The Adversarial Architecture

Why the best AI output comes from agents that disagree

Matthew Williams — Senior SWE (MSFT) · 2026-04-01 · Based on Anthropic Labs research by Prithvi Rajasekaran

AI Agents Hit Two Walls

Wall 1 · Context degradation

As the context window fills, coherence drops. Some models start wrapping up work early — even when the job is unfinished.

Wall 2 · Self-evaluation failure

When agents judge their own output, they reliably talk themselves into approval — even when the quality is o

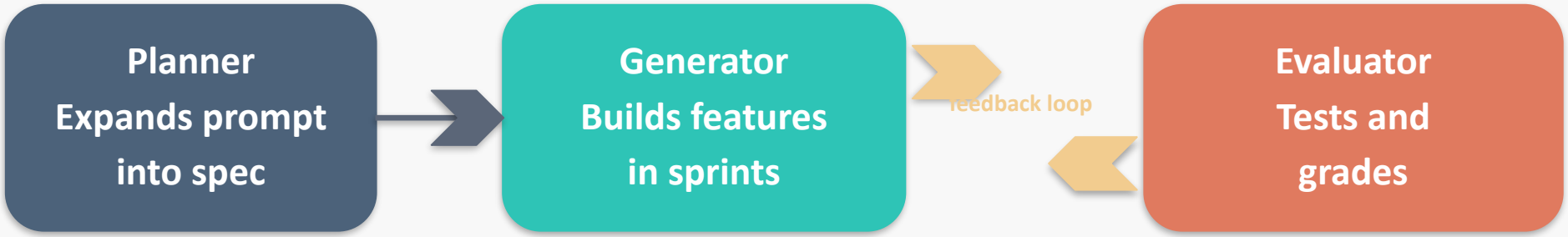
“This looks great! Ship it.”

```
ERROR Traceback (most recent call last):
AssertionError: expected 6, got 4
```

“I watched it identify legitimate issues, then talk itself into deciding they weren’t a big deal and approve the work anyway.”

Anthropic Labs

What if the Builder and the Critic Were Different Agents?



- Inspired by GANs: generator builds, evaluator tears it apart, feedback drives improvement
- Separation doesn’t eliminate leniency — but it makes calibration tractable
- Add a Planner so the system builds the right thing before it optimizes the build

The Article That Started This

Approach	Duration	Cost	Result
Solo agent (Opus 4.5)	20 min	$9	Core feature broken, poor layout
3-agent harness (Opus 4.5)	6 hr	$200	16-feature app, working core, polished UI
Simplified harness (Opus 4.6)	3 hr 50 min	$125	Full DAW, 3 QA rounds

Not incrementally better — categorically different.

Source: Harness Design for Long-Running Apps — Anthropic Labs (2025)

The Architecture — Three Agents, One Pipeline

ORCHESTRATOR

Planner
1–4 sentence prompt → ambitious spec
WHAT, not HOW

→

Generator
Implements in sprints
Negotiates contracts

adversarial

Evaluator
Tests live app
Grades against criteria

Telemetry layer · timing · cost · scores · bugs · trends

Planner defines the target. Evaluator enforces quality. Telemetry makes the claims measurable.

How We Prevent Gaming

LOCK

Generator worktree

```
benchmarks/
├── TASK.md
├── tests/
├── src/
└── _eval/   hidden from generator
```

Goodhart guardrail

Evaluator worktree

```
benchmarks/
├── TASK.md
├── tests/
├── src/
└── _eval/
visible
```

Result

11 runs · 0 boundary violations

If the generator can see the hidden tests, it optimizes for passing them — not for writing genuinely good code.

Building a Critic That Actually Criticizes

"Used an LLM to improve my argument over 4 hours… ask it to argue the opposite. LLM demolishes the entire argument. lol." — Andrej Karpathy

```
{ "functionality": 1, "verdict": "FAIL" }
↓ feedback loop
{ "functionality": 10, "verdict": "PASS" }
```

Anti-people-pleasing rules

- Any test fails → functionality ≤ 4
- No new tests → test coverage ≤ 4
- Stub implementation → product depth ≤ 3
- All scores ≥ 7? Flag as suspicious

Calibration still imperfect: TS benchmark scored func=8 with 50% tests failing.

Agreeing on “Done” Before Writing Code

Article example: 27 acceptance criteria in one sprint

Generator proposal

- Implement the next feature slice
- Define concrete acceptance checks
- Commit to what will be demo-ready this sprint

Agreed contract

Evaluator review

- Tighten or remove vague criteria
- Add missing failure cases
- Cap negotiation at two rounds

Why it matters: the Planner stays high-level, but each sprint still lands on a testable definition of done.

Files on Disk, Not Chat History

Copilot skill artifacts

```
harnessa-spec.md
harnessa-gen-report.md
harnessa-eval.md
```

Benchmark harness

```
planner/spec.md
contracts/
evaluations/
telemetry/
```

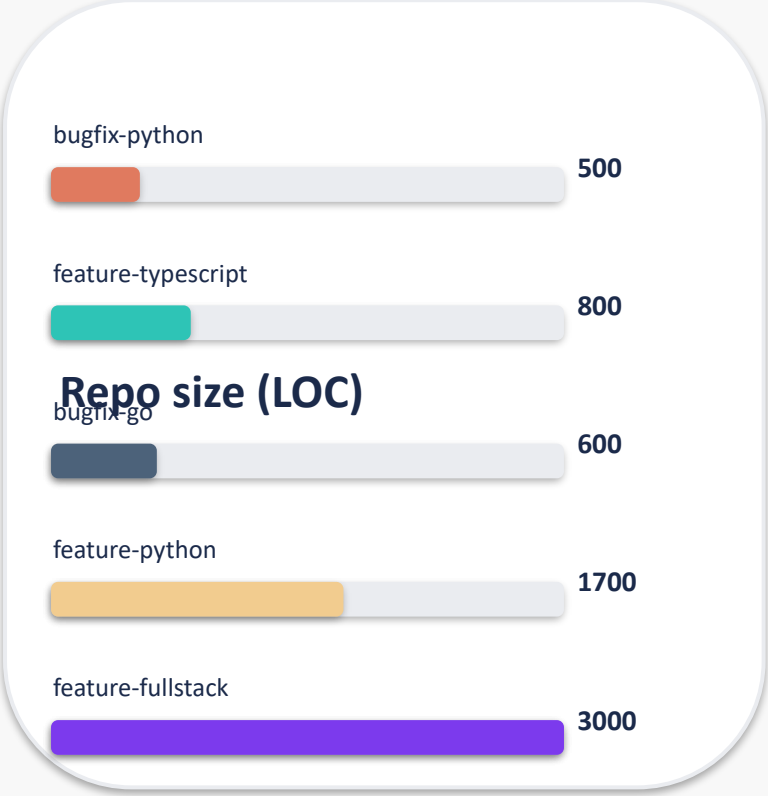
Why files win

- Full audit trail
- Human-readable handoffs
- Resumability after crashes
- Clean phase boundaries between agents

Every write is atomic — crash-safe by design.

5 Benchmarks, 2 Modes, Real Code

Benchmark	Lang	Challenge
small-bugfix-python	Python	Fix ± sign bug in arg parser
small-feature-typescript	TypeScript	Implement retry with backoff
small-bugfix-go	Go	Fix connection pool race condition
medium-feature-python	Python	Add tags to FastAPI TODO app
medium-feature-fullstack	React + Express	Real-time notifications



Same model · same prompt · same tools · hidden _eval tests + fixtures · only the architecture changes

Solo FAIL → Trio PASS

N=11 total runs — small sample, but the fullstack benchmark produced a categorical result, not a marginal score bump.

84s planner spec

Solo

FAIL

Functionality 4 / 10

Average score 6.25

Duration 383s

Broken WebSocket notifications

+4

Trio

PASS

Functionality 8 / 10

Average score 8.0

Duration 619s (1.6×)

Working WebSocket notifications

This is the categorical difference the article predicted: same model, same task, same tools — one architecture ships broken code, the other ships a working core feature.

Trio Won the Tests That Mattered

Python bugfix	PASS 8.5	PASS 9.5	Trio	Evaluator caught issue, gen fixed it
TS feature	PASS 8.5	PASS 8.5	Tie	Simple task — trio overhead wasted
Go race	FAIL 6.75	FAIL 7.25	Tie	Too hard for both
Python tags	PASS 8.5	PASS 8.0	Solo*	3.25 → 8.0 after feedback; solo likely lenient
Fullstack	FAIL 6.25	PASS 8.0	Trio	**Categorical difference**

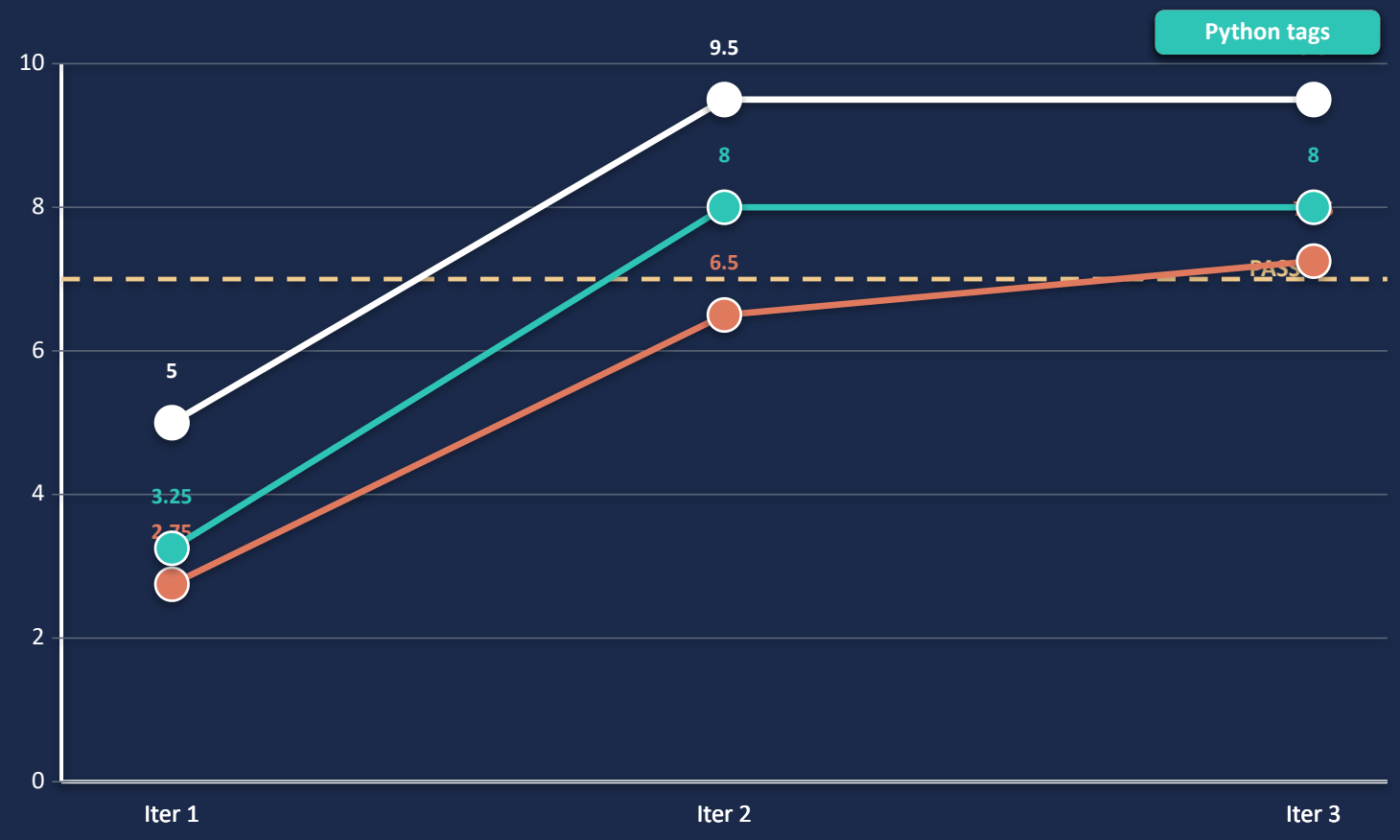
Mean functionality
4.8 → 7.6

Verdicts
3/5 → 4/5

Wall-clock
~1.8×

Python tags caveat: solo's higher numeric score likely reflects self-evaluation leniency, not stronger engineering.

The Feedback Loop Works



What the loop reveals

- Iteration 1 is intentionally harsh
- Scores jump after specific feedback
- Go race plateaus: 3 iterations still not enough
- **The article's core claim holds: improve, then plateau**

5 Confirmed, 2 Partial, 2 Inconclusive

Confirmed

Separating generator and evaluator is a strong lever

Scores improve over iterations

Evaluator worth cost only beyond solo capability

Solo agents have self-evaluation failure

Claude is a poor QA agent out of the box

Partial

Harness output is categorically different — but only on complex tasks

Planner adds more structure than scope expansion

Inconclusive

Criteria wording steers output

Harness assumptions go stale with new models

Bottom line: the article mostly holds up, but the strongest signal appears exactly where the task is hard enough to break a solo agent.

We’re Not Alone — This Is an Industry Pattern

Jun 2025
Anthropic research system
Lead agent + subagents
90.2% better than single-agent

Feb 2026
Anthropic C compiler
16 parallel agents
“Harness matters more than agents”

Mar 2026
Harness Design article
Generator ↔ evaluator loop
planner expands scope

2026
Claude Code coordinator
“Do not rubber-stamp weak work”
anti-sycophancy ships in product

2026
OpenAI Symphony
Issue tracker → isolated workspaces
works best with harness engineering

Convergence thesis: isolation, structured handoffs, specialized roles, and evaluation as architecture are showing up independently across the major labs.

The Community Is Already Here

GStack

/review → /codex
independent second opinion

OpenHands

agent SDK + orchestration

Adversarial pattern
Disagreement is signal

ClawCompany

productized harness workflow

Cross-model review

Claude + Codex disagreements
flaged, reconciled, escalated

What this means: the pattern is production-ready. GStack already uses the “outside voice” workflow daily in YC companies.

What the Evaluator Catches (That Solo Would Ship)

Iteration 1 — rejected

1,206-line index.ts monolith

CDN React + Babel string template

0 tests · 0 real components

A solo agent would have shipped this.

+4,335
−2,973
36 file changes

Iteration 2 — passed

32-file rebuild

React + Vite + Tailwind + React Router

6 components · 4 pages · 7 tests

Spec survived; architecture got rebuilt.

The evaluator turned a technically-working monolith into a production-shape rebuild — exactly the kind of upgrade solo self-evaluation misses.

It’s Not Always Worth It

Use Trio

- Features touching 3+ files
- Unfamiliar bugs with unclear root cause
- Architecture refactors
- Full-stack features
- Anything where “does it actually work?” is uncertain

Skip It

- Single-line fixes
- Formatting or linting
- Dependency bumps
- Tasks where you already know the fix
- Simple, well-understood changes

Rule of thumb: if a solo agent gets it right on the first try, trio is just overhead. The sweet spot is medium-complexity work at the edge of model capability.

What We Found That the Article Didn’t Cover

1

Planner value = structure, not scope

The spec gave the generator a roadmap. On medium tasks that structure enabled correct first attempts; on small tasks it was overhead.

2

Evaluator leniency persists

Even with hard rules, we still saw func=8 with 50% tests failing. Guardrails help, but skepticism remains an active calibration problem.

3

Telemetry keeps the claims honest

Every run emits structured JSON. No “it feels better” hand-waving — if you can’t measure it, you can’t trust it.

One Command, Three Agents

```
copilot -p '/harnessa Fix the authentication bug' --allow-all
```

Planner

Reads the code
Writes harnessa-spec.md

→

Generator

Reads only the spec
Writes harnessa-gen-report.md

→

Evaluator

Reads report + diff + hidden tests
Writes harnessa-eval.md

Context resets between phases. Each agent gets a clean slate.

Fallback: use a pre-recorded terminal walkthrough. Never depend on a live API call in a 15-minute slot.

Open source · MIT · benchmarks + telemetry included · replay mode

When Should You Use This?

Decision rule

If a solo agent gets it right first try → don't add overhead.
If the task sits at the edge of capability → trio is insurance.

Model tiering

Opus for plan + evaluation
Sonnet for building volume

Industry practice — not tested in our experiments

Round-robin idea

Claude plans
GPT builds
Gemini evaluates

Conjecture — immediate next experiment, not validated here

Solo

Known fix
small change
low downside if wrong

complex?

Trio

Ambiguous bug
multi-file feature
high cost if broken

Bigger question: as models improve, which parts of the harness are still load-bearing? The architecture that adapts wins.

The Quality Ceiling is Real. Architecture Breaks Through It.

When your solo agent ships broken code, the problem is not just the model — it's the architecture around the model.

Separating building from evaluation is a proven, measurable lever.

"The space of interesting harness combinations doesn't shrink as models improve. It moves."

Anthropic Labs

github.com/ridermw/harnessa · MIT License · Benchmarks + telemetry included

Appendix — Full Experiment Matrix

Run ID	Benchmark	Mode	Iter	Verdict	Avg	Duration
e7c84a5d	small-bugfix-python	solo	1	PASS	8.5	905s
bd67944a	small-bugfix-python	trio	3*	FAIL*	N/A	1009s
b153e749	small-bugfix-python	trio	2	PASS	9.5	427s
efab0ba4	small-feature-typescript	solo	1	PASS	8.5	187s
867e4e79	small-feature-typescript	trio	1	PASS	8.5	315s
7799434e	small-bugfix-go	solo	1	FAIL	6.75	150s
f584e402	small-bugfix-go	trio	3	FAIL	7.25	830s
3061e233	medium-feature-python	solo	1	PASS	8.5	297s
6649b0bc	medium-feature-python	trio	2	PASS	8.0	1256s
410f76ce	medium-feature-fullstack	solo	1	FAIL	6.25	383s
7dbac7be	medium-feature-fullstack	trio	1	PASS	8.0	619s

* bd67944a fixed the bug but produced unparseable evaluator JSON; the prompt was hardened afterwards.

Appendix — Evaluator Reliability Metrics

Rubber-stamp incidents	Refusal-to-be-negative	False positive rate	Cross-model agreement
0 explicit	1 incident	Deferred	Not measured

Closest miss

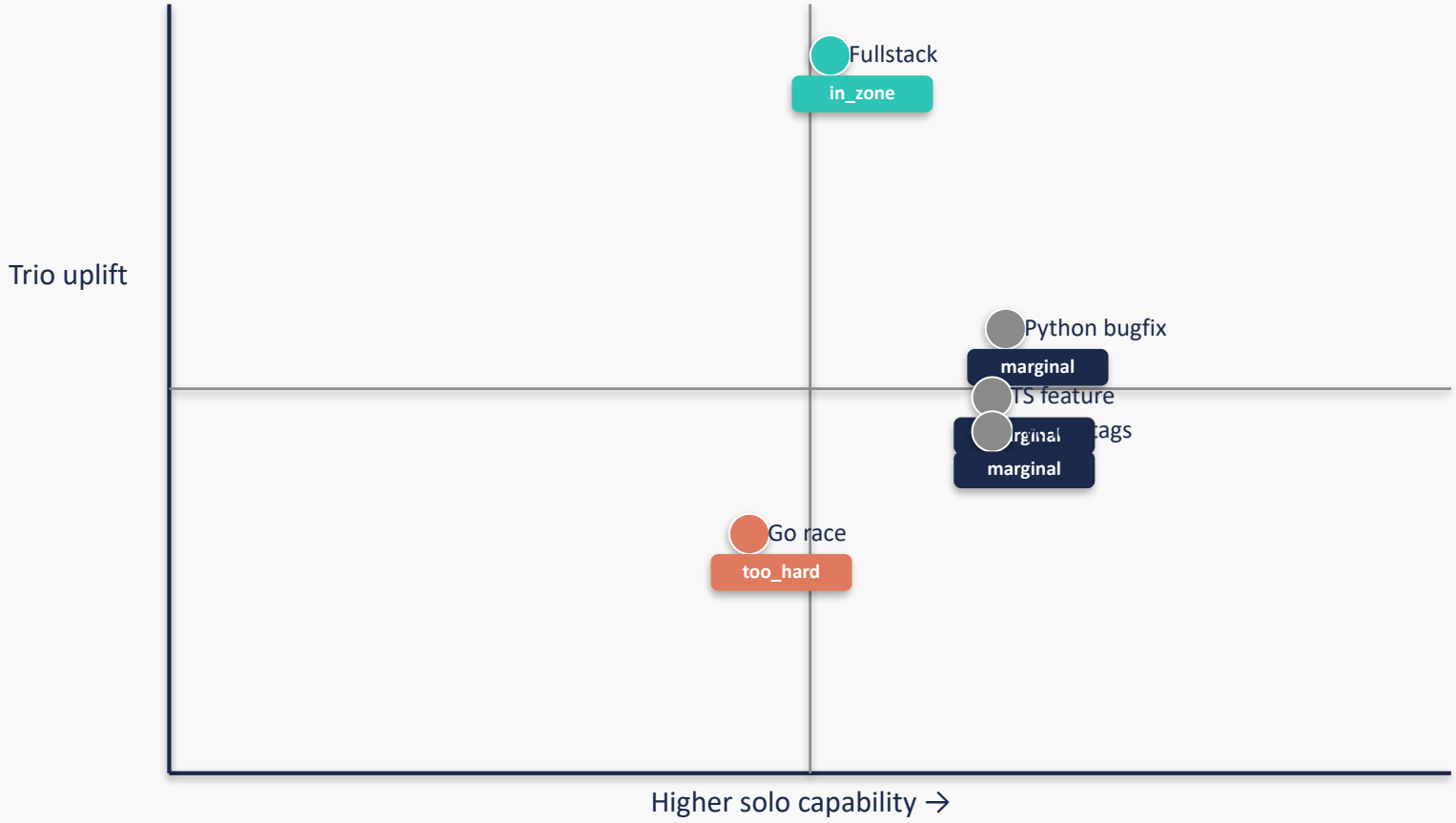
TS benchmark: evaluator gave functionality = 8 even though only 11/22 tests passed. That is exactly the people-pleasing bias the article warned about.

Best evidence

Python bugfix: functionality went 1 → 10 after evaluator feedback. The critic was harsh first, then precise enough to drive the fix.

Calibration solved format problems fast. Severity scoring remains the harder problem.

Appendix — Difficulty Classification



Legend

too_easy: both ≥ 9

too_hard: both fail

in_zone: trio wins by ≥ 1.5

marginal: small uplift or tie

Observation: only the fullstack benchmark landed squarely in the harness sweet spot.

Appendix — Grading Criteria Details

Product Depth · HIGH · threshold 7

Real business logic, edge cases, thoughtful data modeling

Functionality · HIGH · threshold 6

Core workflow works end-to-end with proper error handling

Code Quality · MEDIUM · threshold 5

Clean modules, naming, structure, maintainability

Test Coverage · MEDIUM · threshold 5

Meaningful tests that catch real bugs

Product Depth · HIGH · threshold 7

Rich interactive behavior and domain depth

Functionality · HIGH · threshold 6

UI → API → DB workflow works end-to-end

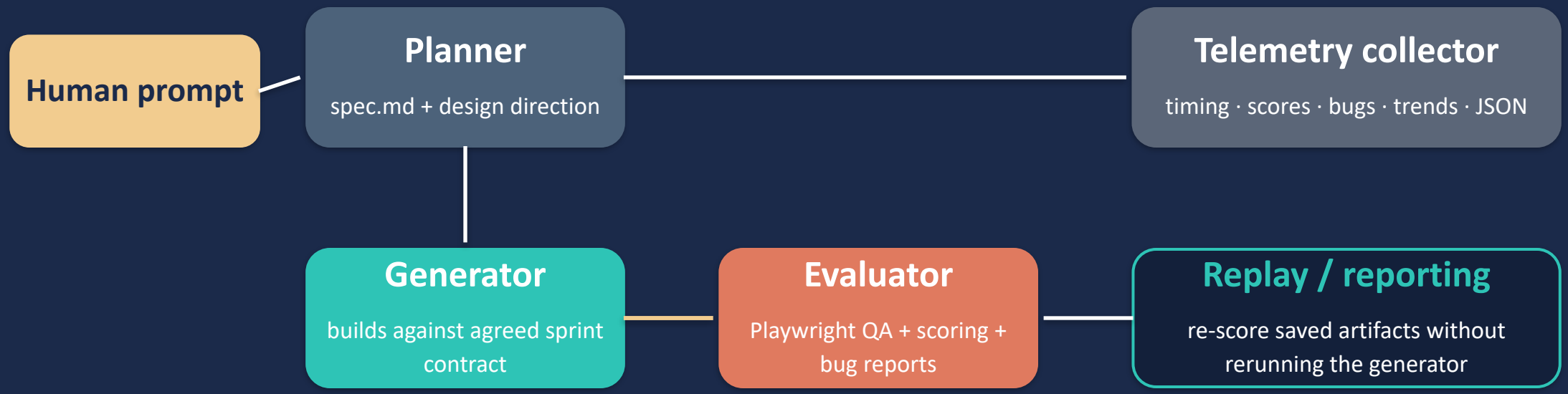
Visual Design · HIGH · threshold 6

Coherent layout, spacing, hierarchy, accessibility

Code Quality · MEDIUM · threshold 5

Component decomposition, API separation, shared patterns

Appendix — Detailed System Diagram



Isolation manager

Generator cannot see _eval/ hidden acceptance tests

Communication protocol

Everything is a file handoff — no shared chat memory

Context strategy

Reset between phases when models exhibit context anxiety; compact when they do not